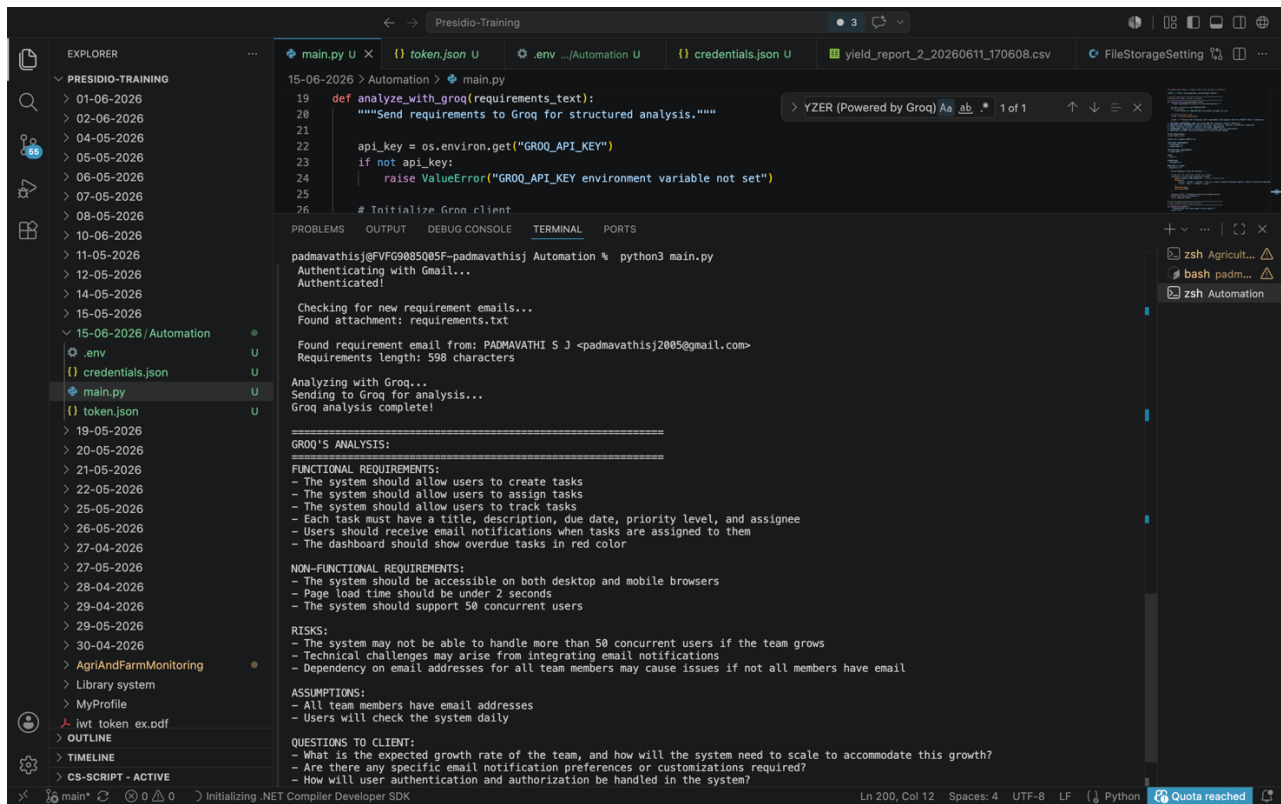


AI Agent Assignment

GitHub Source Code URL - <https://github.com/Padmavathi-SJ/Presidio-training/tree/main/15-06-2026/Automation>

Outputs:



The screenshot displays a VS Code editor window titled "Presidio-Training". The Explorer sidebar on the left shows a directory structure with folders for dates from 01-06-2026 to 30-04-2026, and a folder named "Automation" which is currently selected. Inside "Automation", files like ".env", "credentials.json", "main.py", and "token.json" are listed. The main editor area shows the "main.py" file with the following Python code:

```
19 def analyze_with_groq(requirements_text):
20     """Send requirements to Groq for structured analysis."""
21
22     api_key = os.getenv("GROQ_API_KEY")
23     if not api_key:
24         raise ValueError("GROQ_API_KEY environment variable not set")
25
26     # Initialize Groq client
```

Below the code editor, the TERMINAL panel shows the execution output of the command `python3 main.py`. The output includes authentication steps, email checking, and a detailed analysis from Groq. The analysis is categorized into Functional Requirements, Non-Functional Requirements, Risks, Assumptions, and Questions to Client.

```
padmavathisj@FVFG9085Q85F-padmavathisj Automation % python3 main.py
Authenticating with Gmail...
Authenticated!

Checking for new requirement emails...
Found attachment: requirements.txt

Found requirement email from: PADMAVATHI S J <padmavathisj2005@gmail.com>
Requirements length: 598 characters

Analyzing with Groq...
Sending to Groq for analysis...
Groq analysis complete!

=====
GROQ'S ANALYSIS:
=====
FUNCTIONAL REQUIREMENTS:
- The system should allow users to create tasks
- The system should allow users to assign tasks
- The system should allow users to track tasks
- Each task must have a title, description, due date, priority level, and assignee
- Users should receive email notifications when tasks are assigned to them
- The dashboard should show overdue tasks in red color

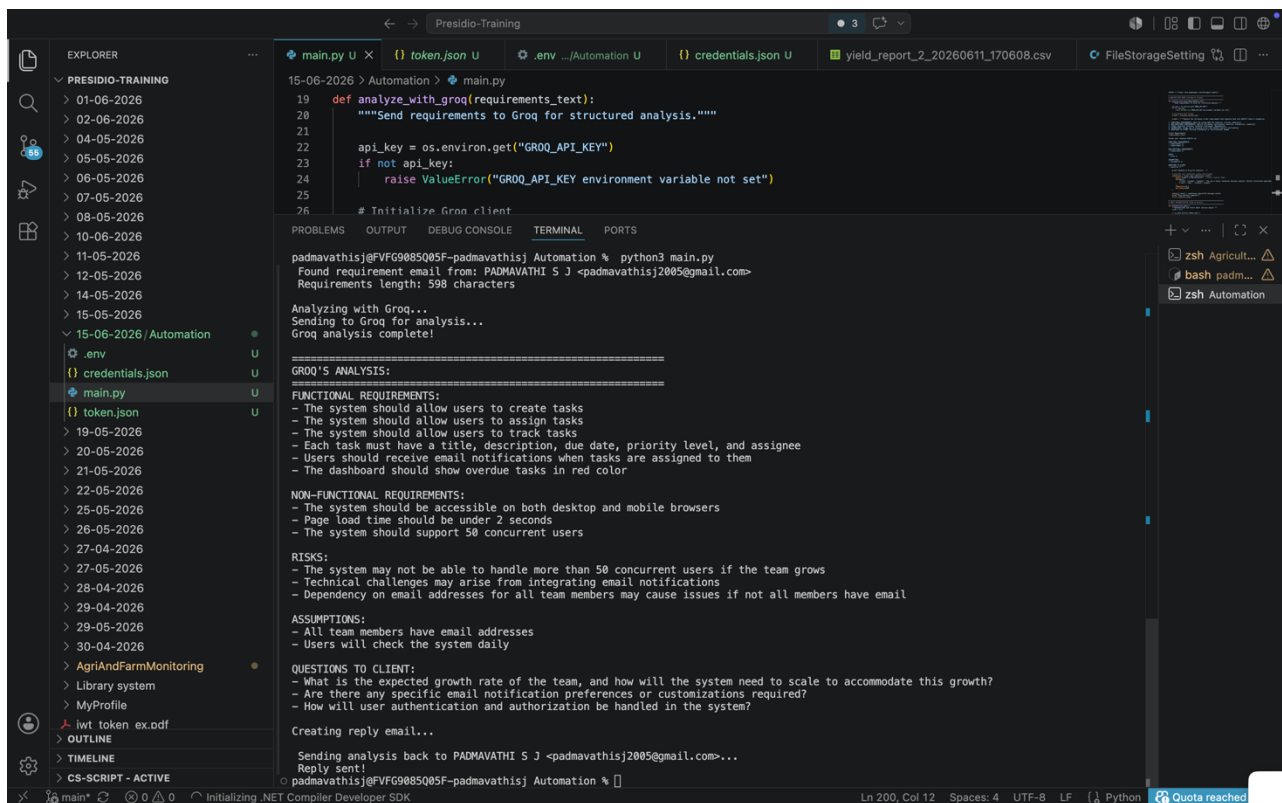
NON-FUNCTIONAL REQUIREMENTS:
- The system should be accessible on both desktop and mobile browsers
- Page load time should be under 2 seconds
- The system should support 50 concurrent users

RISKS:
- The system may not be able to handle more than 50 concurrent users if the team grows
- Technical challenges may arise from integrating email notifications
- Dependency on email addresses for all team members may cause issues if not all members have email

ASSUMPTIONS:
- All team members have email addresses
- Users will check the system daily

QUESTIONS TO CLIENT:
- What is the expected growth rate of the team, and how will the system need to scale to accommodate this growth?
- Are there any specific email notification preferences or customizations required?
- How will user authentication and authorization be handled in the system?
```

The status bar at the bottom indicates the file is at Line 200, Column 12, using UTF-8 encoding and LF line endings. A "Quota reached" warning is also visible on the right side of the status bar.



```
15-06-2026 > Automation > main.py
19 def analyze_with_groq(requirements_text):
20     """Send requirements to Groq for structured analysis."""
21
22     api_key = os.environ.get("GROQ_API_KEY")
23     if not api_key:
24         raise ValueError("GROQ_API_KEY environment variable not set")
25
26     # Initialize Groq client
27
28     groq = Groq(api_key=api_key)
29
30     # Send requirements to Groq for analysis
31     analysis = groq.parse_text(requirements_text)
32
33     # Send analysis back to the email address
34     email_address = find_email(requirements_text)
35     if email_address:
36         send_email(email_address, analysis)
37
38     return analysis
39
40 if __name__ == "__main__":
41     requirements_text = read_requirements()
42     analysis = analyze_with_groq(requirements_text)
43     print(analysis)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

padmavathisj@FVFG9085Q05F-padmavathisj Automation % python3 main.py
Found requirement email from: PADMAVATHI S J <padmavathisj2005@gmail.com>
Requirements length: 598 characters

Analyzing with Groq...
Sending to Groq for analysis...
Groq analysis complete!

GROQ'S ANALYSIS:

FUNCTIONAL REQUIREMENTS:

- The system should allow users to create tasks
- The system should allow users to assign tasks
- The system should allow users to track tasks
- Each task must have a title, description, due date, priority level, and assignee
- Users should receive email notifications when tasks are assigned to them
- The dashboard should show overdue tasks in red color

NON-FUNCTIONAL REQUIREMENTS:

- The system should be accessible on both desktop and mobile browsers
- Page load time should be under 2 seconds
- The system should support 50 concurrent users

RISKS:

- The system may not be able to handle more than 50 concurrent users if the team grows
- Technical challenges may arise from integrating email notifications
- Dependency on email addresses for all team members may cause issues if not all members have email

ASSUMPTIONS:

- All team members have email addresses
- Users will check the system daily

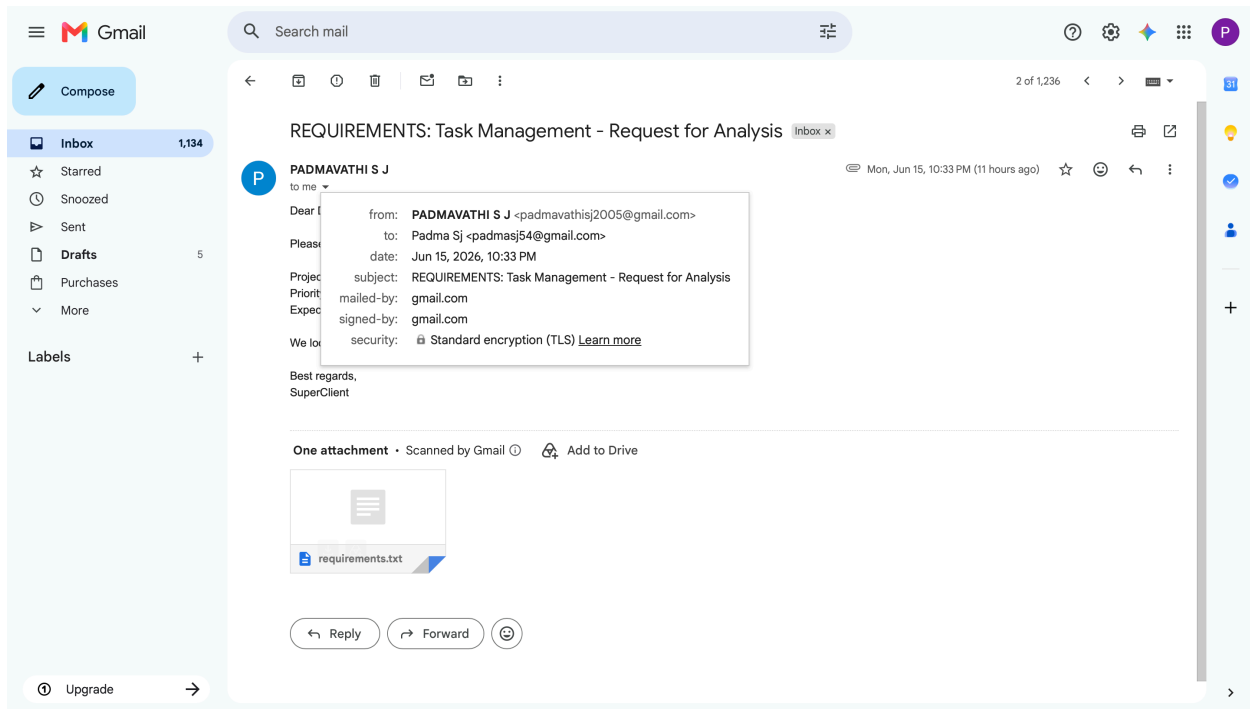
QUESTIONS TO CLIENT:

- What is the expected growth rate of the team, and how will the system need to scale to accommodate this growth?
- Are there any specific email notification preferences or customizations required?
- How will user authentication and authorization be handled in the system?

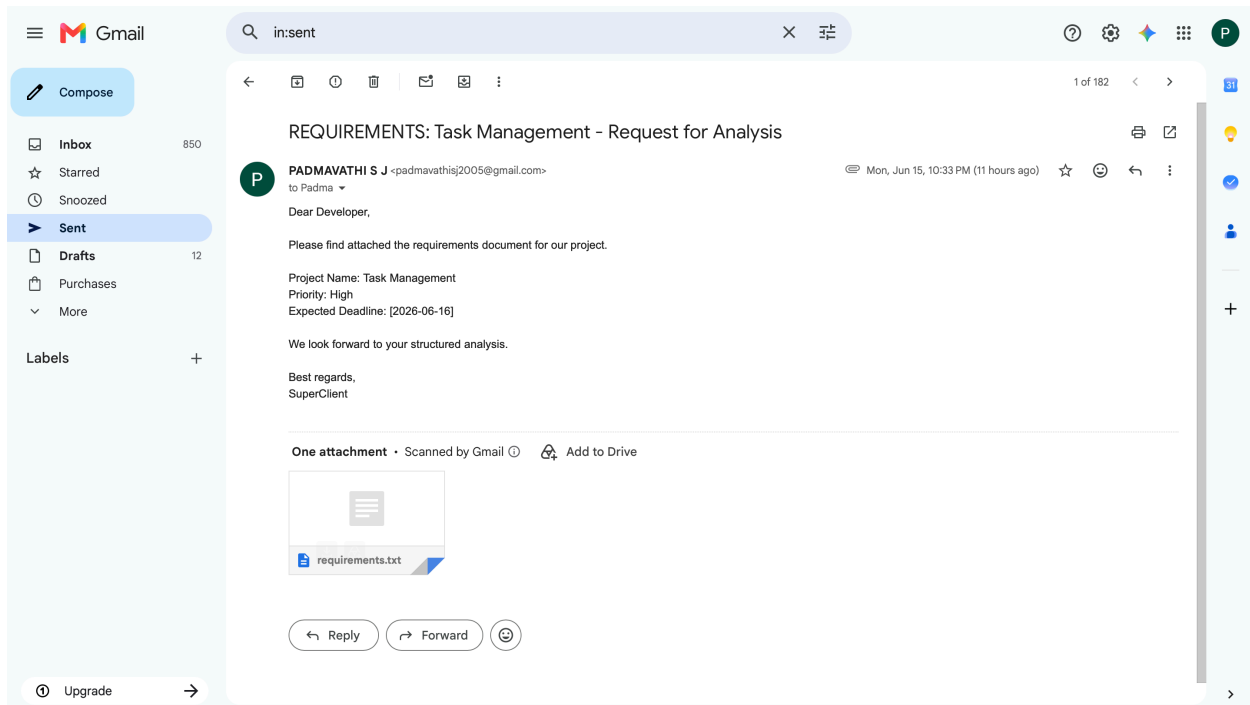
Creating reply email...
Sending analysis back to PADMAVATHI S J <padmavathisj2005@gmail.com>...
Reply sent!

padmavathisj@FVFG9085Q05F-padmavathisj Automation %

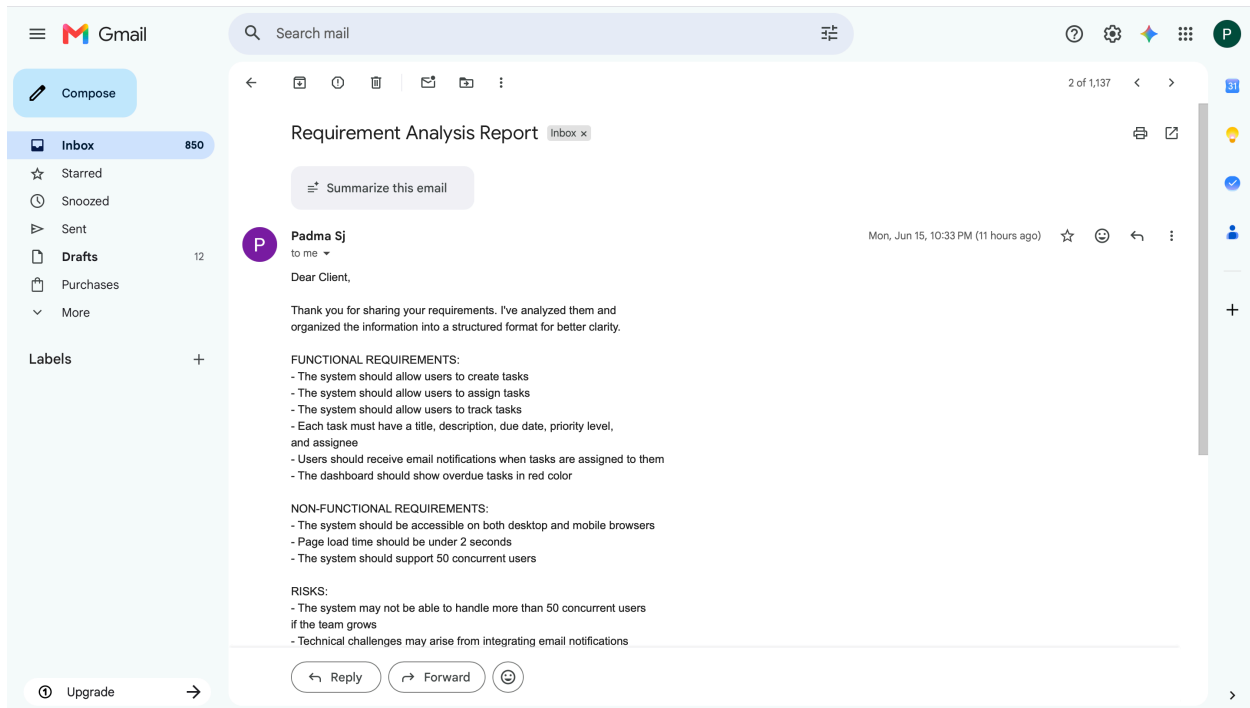
Client sent email to Developer(AI Automation Bot Email):

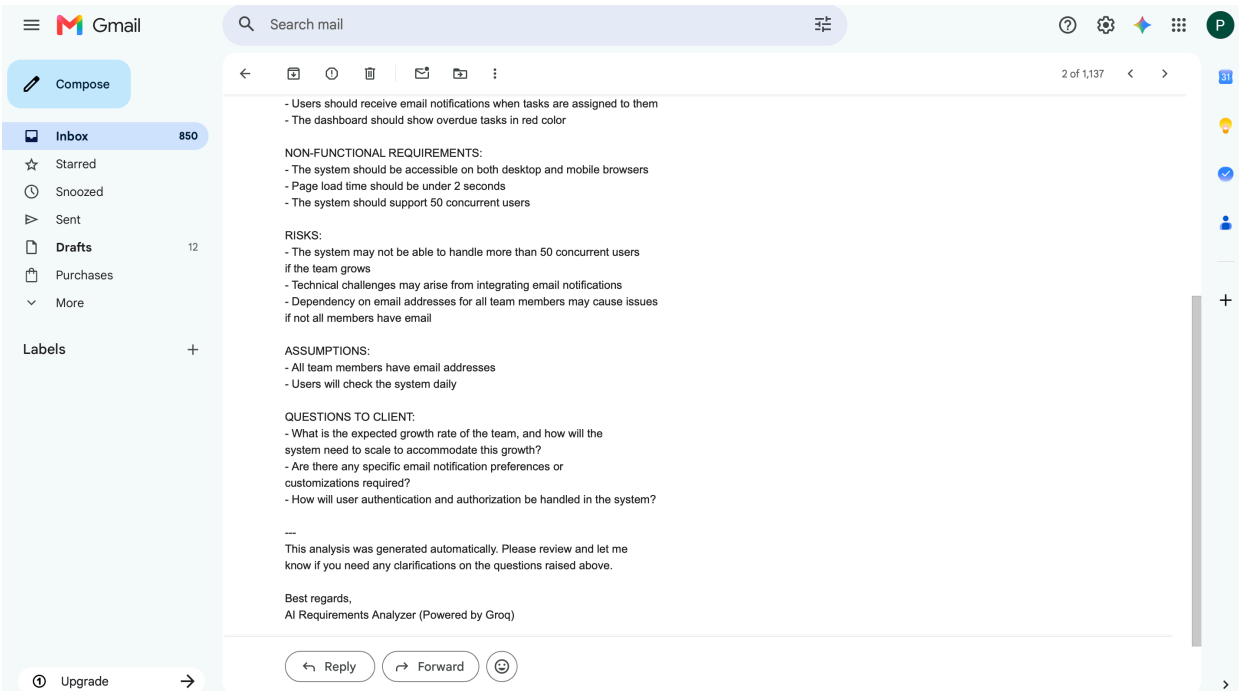


Email Received from client:



After analysing reply email received from AI Bot to client





1. Prompt Used

System Prompt (for Groq):

You are a senior technical business analyst. Extract structured requirements.

User Prompt:

Analyze the following client requirements and organize them into EXACTLY these 5 categories:

1. FUNCTIONAL REQUIREMENTS: What the system MUST DO (features, actions, behaviors)
2. NON-FUNCTIONAL REQUIREMENTS: Quality attributes (performance, security, scalability, usability)
3. RISKS: Potential problems, technical challenges, dependencies
4. ASSUMPTIONS: Things you're assuming to be true without explicit confirmation
5. QUESTIONS TO CLIENT: Missing information or clarifications needed

2. Client Requirements:

{requirements_text}

Format your response EXACTLY as:

FUNCTIONAL REQUIREMENTS:

- [requirement 1]
- [requirement 2]

NON-FUNCTIONAL REQUIREMENTS:

- [requirement 1]

RISKS:

- [risk 1]

ASSUMPTIONS:

- [assumption 1]

QUESTIONS TO CLIENT:

- [question 1]

3. Groq Conversation:

What is sent to Groq:

- System prompt: "You are a senior technical business analyst"
- User prompt: Requirements text + 5 categories instruction
- Model: llama-3.3-70b-versatile
- Temperature: 0.3 (consistent output)

What Groq returns:

- Functional Requirements
- Non-functional Requirements
- Risks
- Assumptions
- Questions to Client

API Details:

- Endpoint: `POST /v1/chat/completions`
- Authentication: Bearer token (GROQ_API_KEY)
- Response time: 200-500ms
- Token usage: ~770 tokens per analysis

Code Flow:

1. `client.chat.completions.create()`
2. Pass messages array (system + user)
3. Receive structured analysis
4. Extract from `choices[0].message.content`

4. Sample Input (requirements.txt)

- Build a customer support chatbot for our e-commerce website.
- The chatbot should answer common questions about orders, shipping, and returns.
- It needs to integrate with our existing inventory database to check product availability.
- The system should support English and Spanish languages.
- We expect around 500 concurrent users during peak shopping seasons.
- Security is critical since customers may share order numbers and personal info.
- The response time should be under 2 seconds.

- The system must be available 24/7 with 99.9% uptime.
- All conversations should be logged for quality analysis.
- We assume the inventory API is well-documented.
- We assume customers have basic internet connectivity.

5. Sample Output (Groq's Analysis)

FUNCTIONAL REQUIREMENTS:

- Answer common questions about orders, shipping, and returns
- Integrate with inventory database for product availability
- Support English and Spanish languages
- Log all conversations for quality analysis

NON-FUNCTIONAL REQUIREMENTS:

- Response time under 2 seconds
- 99.9% uptime (24/7 availability)
- Secure handling of order numbers and personal info
- Support 500 concurrent users during peak seasons

RISKS:

- Inventory API may have rate limits or downtime
- Language translation accuracy for Spanish
- Security vulnerabilities if customer data is not properly encrypted
- Scalability challenges during unexpected traffic spikes

ASSUMPTIONS:

- Inventory API is well-documented and stable
- Customers have basic internet connectivity
- Customer consent is obtained for conversation logging

QUESTIONS TO CLIENT:

- What is the expected conversation volume per day?
- Should the chatbot escalate to human agents? If yes, when?
- What is the retention policy for conversation logs?
- Are there compliance requirements (GDPR, CCPA, etc.)?

6. ARCHITECTURE DIAGRAM:

